Script-Controlled ACL: Restrict Record Access Based on Field Value

Requirement Analysis: Customer Journey Map

- **Project Name:** Script-Controlled ACL: Restrict Record Access Based on Field Value
- **Scenario:** Creating users, roles and a custom table in ServiceNow, then protecting student records with script-controlled READ, CREATE, WRITE and DELETE ACLs and verifying the access for each user type.

Customer Journey Stages

1. Entice (Awareness & Need)

- **What people experience:** The administrator realizes that sensitive student records in ServiceNow are visible to every user with basic access, and that records must be restricted by branch (for example, EEE).
- **Interactions (Touchpoints):** Security requirements, institution data-protection policy, and discussions with faculty and branch coordinators.
- **Goals & Motivations:** “Help me find out how to control who can see and change each student record.”
- **Positive Moments:** Realizing that ServiceNow Access Control Lists (ACLs) can enforce rules using both roles and field values.
- **Negative Moments:** Uncertainty about which operations (read, create, write, delete) need protection and where to start.
- **Areas of Opportunity:** Document clear access rules per role (bb1 to bb4) before building, so every ACL has a defined purpose.

2. Enter (Discovery & Initiation)

- **What people experience:** The administrator logs in with admin access, creates the EEE test user and the custom roles bb1 to bb4, and creates the Institution Details table (u_institution_details) with the Branch field and sample records.
- **Interactions (Touchpoints):** User Administration (Users and Roles), System Definition (Tables), and the Institution Details list.
- **Goals & Motivations:** “Help me set up users, roles and data correctly so the ACLs can be tested.”
- **Positive Moments:** Clear, organized ServiceNow forms make it easy to create the user, roles, table and records for ECE, EEE and CSE branches.
- **Negative Moments:** Missing a role assignment or a field setting can cause the ACLs to behave unexpectedly during testing.
- **Areas of Opportunity:** Use a checklist to confirm that all four roles are assigned to the EEE user and that records exist for every branch.

3. Engage (Core Process: ACL Creation)

- **What people experience:** The administrator elevates to security_admin and creates four ACLs on u_institution_details: READ (role bb1, data condition Branch is EEE, advanced script), CREATE (bb2), WRITE (bb3) and DELETE (bb4).

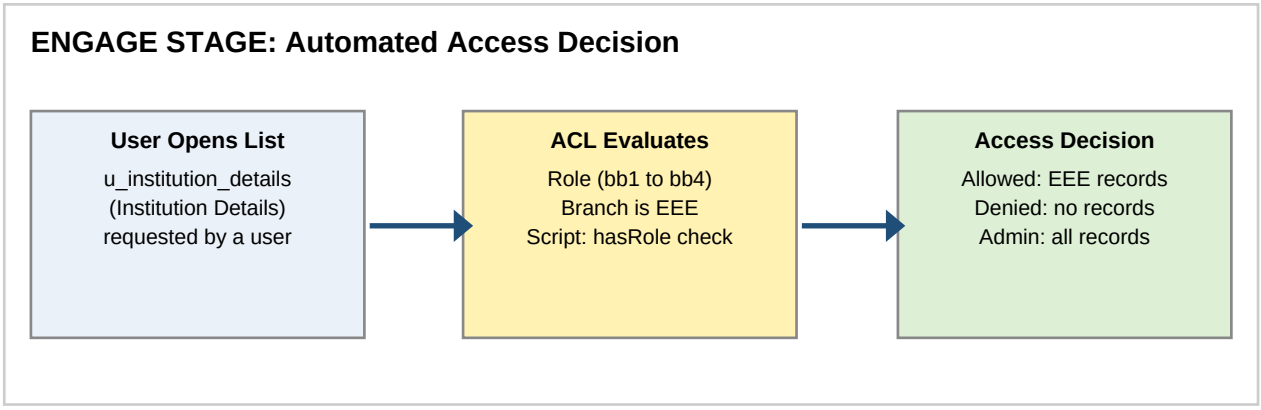
- **Interactions (Touchpoints):** Elevate Role dialog, System Security > Access Control (ACL) list, ACL form with Requires role related list, data condition and script editor.
- **Goals & Motivations:** “Help me make sure only the right roles can read, create, edit and delete the right records.”
- **Positive Moments:** Script-controlled ACLs are evaluated automatically on every access, so no manual checking is needed and administrators keep full access.
- **Negative Moments:** A wrong script or missing role in an ACL can block valid users or expose records to the wrong users.
- **Areas of Opportunity:** Test each ACL immediately after saving, and keep the script simple and commented so it is easy to review.

4. Exit (Verification & Validation)

- **What people experience:** The administrator impersonates the EEE user, a user without roles, and the admin, then opens the Institution Details list to confirm what each can see and do.
- **Interactions (Touchpoints):** Impersonate User option, the u_institution_details list view, the New button, inline editing, and the Delete action.
- **Goals & Motivations:** “Help me confirm that the access rules work exactly as designed.”
- **Positive Moments:** A bb1 user sees only EEE records, a user without roles sees none, and an admin sees all records. With bb1 to bb4, the user can also create, edit and delete EEE records.
- **Negative Moments:** If a role is missing, the New button, inline edit or Delete action does not appear, which can be confusing during testing.
- **Areas of Opportunity:** Record each test result and screenshot, so the verification evidence is ready for review.

5. Extend (Post-Setup & Review)

- **What people experience:** Authorized users work only with the records they are permitted to access, and the administrator has confidence that student data is protected.
- **Interactions (Touchpoints):** ACL records, user and role reviews, and feedback from faculty and branch users.
- **Goals & Motivations:** “Help me keep student data protected as new users and branches are added.”
- **Positive Moments:** Secure, consistent record-level access that builds trust in how the institution handles student data.
- **Negative Moments:** None, once ACLs are tested and roles are assigned correctly. New branches will need new rules.
- **Areas of Opportunity:** Extend the same pattern to the ECE and CSE branches and review roles regularly.

Journey Map Grid Matrix

KEY VECTORS	1. ENTICE Discover	2. ENTER Setup	3. ENGAGE Create ACLs	4. EXIT Verify	5. EXTEND Review
USER ACTIONS	Identifying sensitive student data	Creating user, roles, table and records	Writing READ, CREATE, WRITE, DELETE ACLs	Impersonating users and checking lists	Reviewing roles, adding branches
TOUCHPOINTS	Policy, branch coordinators	Users, Roles, Tables modules	Elevate role, ACL form, script editor	Impersonate user, list view	ACL list, user feedback
THOUGHTS & EMOTIONS	Concern, curiosity	Focus, careful	Confident, cautious	Relief, satisfaction	Trust, confidence
PAIN POINTS	Unclear where to start	Missed role or field setup	Wrong script blocks users	Missing buttons confuse testers	New branches need new rules